Лабораторная работа №7. Коллективные операции MPI

Выполнил: студент группы 11-404 Заика Тимур

1. Реализация и система

Реализована программа `mpi_array_sum.c` на языке C для распределения массива между MPI-процессами. Вычисление локальных сумм и сбор результата выполняются тремя способами: через `MPI_Gather` (с ручным суммированием на корне), через `MPI_Reduce` и через `MPI_Allreduce`.

Измерения проводились с помощью `MPI_Wtime`. Для каждой конфигурации выполнялось 5 запусков, вычислялось среднее время.
Тестовая среда: Apple MacBook Air (Apple Silicon), OpenMPI, GCC (Clang).

2. Результаты

Таблица 1. Проверка корректности (P = 4)

N	Ожидаемая сумма	Gather	Reduce	Allreduce	Корректно?
100	5050	5050	5050	5050	Да
10 000	50005000	50005000	50005000	50005000	Да
1 000 000	500000500000	500000500000	500000500000	500000500000	Да
10 000 000	50000005000000	50000005000000	50000005000000	50000005000000	Да

Таблица 2. Влияние размера массива на время (P = 4)

N	Время выполнения, с	Результат корректен?
10 000	0.0003	Да
100 000	0.0010	Да
1 000 000	0.0092	Да
10 000 000	0.0874	Да

Таблица 3. Влияние числа процессов (N = 10 000 000)

Число процессов P	Время T_P, с	Ускорение S_P	Эффективность E_P
1	0.2850	1.00	1.00
2	0.1502	1.90	0.95
4	0.0874	3.26	0.82
8	0.0950	3.00	0.38

Таблица 4. Сравнение Gather и Reduce (N = 1 000 000, 10000 повторений)

Число процессов P	Способ агрегирования	Среднее время, с	Корректно?
4	Gather + сумма на процессе 0	0.0000045	Да
4	Reduce	0.0000031	Да
8	Gather + сумма на процессе 0	0.0000068	Да
8	Reduce	0.0000042	Да

3. Вывод

В ходе работы были изучены коллективные операции MPI. Все три способа агрегации (`Gather`, `Reduce`, `Allreduce`) показали корректный результат, совпадающий с формулой N(N+1)/2.

Анализ Таблицы 3 показывает, что при увеличении числа процессов до 4 наблюдается хорошее ускорение (в 3.26 раза при эффективности 82%). Однако при переходе на 8 процессов время возрастает, а эффективность падает до 38%. Это связано с тем, что накладные расходы на пересылку данных и синхронизацию начинают превышать время полезных вычислений.

Сравнение `MPI_Gather` и `MPI_Reduce` показывает, что `Reduce` работает быстрее. Это ожидаемо: `Gather` собирает массив локальных сумм на корневом процессе, после чего тот вынужден выполнять последовательный цикл сложения. `Reduce` же выполняет свертку оптимизированно, избавляя корневой процесс от лишней работы.

Разница между операциями заключается в их назначении: **Gather** собирает все отдельные значения на корневом процессе; **Reduce** возвращает свернутый результат только корневому процессу; **Allreduce** возвращает свернутый результат всем процессам, что удобно, если итоговая сумма нужна каждому участнику для дальнейших вычислений.